

# **Fundamentos de Algoritmos y Estructuras de Datos (fuente propia)**

## 1. Complejidad algorítmica + ¿Qué es un algoritmo? + ¿Qué es complejidad algorítmica? + Notación Big O

### ¿Qué es un algoritmo?

Es una serie de pasos para resolver un problema.

Problema: ordenar una lista de números  
Algoritmo: compara, intercambia, repite

### ¿Qué es complejidad algorítmica?

Es medir **cuantos pasos** necesita un algoritmo según el tamaño del input **n**.

No se mide en segundos – porque depende del computador. Se mide en **pasos**.

### Notación Big O

Se escribe **O(algo)** y describe como crecen los pasos cuando n crece.

| Notación      | Nombre             | Ejemplo real                      |
|---------------|--------------------|-----------------------------------|
| $O(1)$        | Constante          | Acceder a un elemento de un array |
| $O(\log n)$   | Logarítmica        | Búsqueda binaria                  |
| $O(n)$        | Lineal             | Recorrer un array                 |
| $O(n \log n)$ | Lineal-logarítmica | Merge Sort                        |
| $O(n^2)$      | Cuadrática         | Bubble Sort                       |

| Notacion      | Nombre             | Ejemplo real                      |
|---------------|--------------------|-----------------------------------|
| $O(1)$        | Constante          | Acceder a un elemento de un array |
| $O(\log n)$   | Logaritmica        | Busqueda binaria                  |
| $O(n)$        | Lineal             | Recorrer un array                 |
| $O(n \log n)$ | Lineal-logaritmica | Merge Sort                        |
| $O(n^2)$      | Cuadratica         | Buble Sort                        |

1.1 Ejemplos en código Simple (O(1) – O(n) – O(n²) + Tabla Comparativa

php

// 0(1) – siempre 1 paso  
\$arr = [1, 2, 3, 4, 5];  
echo \$arr[0];  
  
// 0(n) – 1 paso por elemento  
foreach (\$users as \$user) {  
 echo \$user->name;  
}  
  
// 0(n²) – loop dentro de loop  
foreach (\$arr as \$i) {  
 foreach (\$arr as \$j) {  
 // n \* n pasos  
 }  
}

Tabla Comparativa

| Notación   | 10 elementos | 1.000 elementos | 1.000.000 elementos |
|------------|--------------|-----------------|---------------------|
| O(1)       | 1 paso       | 1 paso          | 1 paso              |
| O(log n)   | 3 pasos      | 10 pasos        | 20 pasos            |
| O(n)       | 10 pasos     | 1.000 pasos     | 1.000.000 pasos     |
| O(n log n) | 33 pasos     | 10.000 pasos    | 20.000.000 pasos    |
| O(n²)      | 100 pasos    | 1.000.000 pasos | 💀                   |

| Notacion    | 10 elementos | 1.000 elementos | 1.000.000 elementos |
|-------------|--------------|-----------------|---------------------|
| O(1)        | 1 paso       | 1 paso          | 1 paso              |
| O (log n)   | 3 pasos      | 10 pasos        | 20 pasos            |
| O (n)       | 10 pasos     | 1.000 pasos     | 1.000.000 pasos     |
| O (n log n) | 33 pasos     | 10.000 pasos    | 20.000.000 pasos    |
| O(n²)       | 100 pasos    | 1.000.000 pasos | 💀                   |

### 1.1.1 O (log n) - ¿Por qué 3 pasos con 10 elementos? (Objetivo: llegar a 1)

| Notación   | 10 elementos | 1.000 elementos | 1.000.000 elementos |
|------------|--------------|-----------------|---------------------|
| O(1)       | 1 paso       | 1 paso          | 1 paso              |
| O(log n)   | 3 pasos      | 10 pasos        | 20 pasos            |
| O(n)       | 10 pasos     | 1.000 pasos     | 1.000.000 pasos     |
| O(n log n) | 33 pasos     | 10.000 pasos    | 20.000.000 pasos    |
| O(n²)      | 100 pasos    | 1.000.000 pasos | 💩                   |

La pregunta que responde log n es:

¿Cuántas veces dividido n entre 2 hasta llegar a 1?

Con n = 10:

```
10 ÷ 2 = 5
5 ÷ 2 = 2.5
2.5 ÷ 2 = 1.25 ≈ 1 ✓
```

3 divisiones -> **3 pasos**

Con n =1.000

```
1000 ÷ 2 = 500
500 ÷ 2 = 250
250 ÷ 2 = 125
125 ÷ 2 = 62
62 ÷ 2 = 31
31 ÷ 2 = 15
15 ÷ 2 = 7
7 ÷ 2 = 3
3 ÷ 2 = 1 ✓
```

9 divisiones -> **~10 pasos**

Con n = 1.000.000:

```
Divides 20 veces hasta llegar a 1
porque 2²⁰ = 1.048.576 ≈ 1.000.000
```

→ **~20 pasos**

### 1.1.1.1 Recordatorio de Logaritmo + Ejemplos concretos + ¿Por qué base 2 en logaritmos?

#### ¿Qué es un logaritmo?

Es la operación inversa de la potencia.

Si la potencia pregunta:

$$2^3 = ? \rightarrow 2 \times 2 \times 2 = 8$$

El logaritmo pregunta al revés:

¿A qué potencia elevo 2 para obtener 8?  
 $\log_2(8) = 3$

#### La fórmula general

Si:  $b^x = n$   
Entonces:  $\log_b(n) = x$

Donde **b** es la **base** y **x** es el resultado

#### Ejemplos concretos

$\log_2(8) = 3$  porque  $2^3 = 8$   
 $\log_2(16) = 4$  porque  $2^4 = 16$   
 $\log_2(32) = 5$  porque  $2^5 = 32$   
 $\log_{10}(100) = 2$  porque  $10^2 = 100$   
 $\log_{10}(1000) = 3$  porque  $10^3 = 1000$

#### ¿Por qué base 2 en algoritmos?

Porque los algoritmos dividen entre 2 en cada paso – como la búsqueda binaria. Entonces la pregunta natural es:

¿Cuántas veces dividido entre 2 hasta llegar a 1?

**Con n = 10:**

$$\log_2(10) = 3.32 \approx 3 \text{ pasos}$$

**Con n = 1.000:**

$$\log_2(1000) = 9.96 \approx 10 \text{ pasos}$$

**Con n = 1.000.000:**

$$\log_2(1.000.000) = 19.93 \approx 20 \text{ pasos}$$

### 1.1.1.2 ¿Cómo se calcula a mano? + La conexión con Big O + El 1 representa un solo elemento

Preguntas: ¿Cuántas veces multiplico la base por si misma para llegar a n?

$\log_2(32) = ?$



$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$  ✓

$\log_2(32) = 5$

### La conexión con Big O

Cuando ves  $O(\log n)$  en algoritmos, siempre es log base 2 – porque cada paso descarta **la mitad** de los datos.

1.000.000 elementos



→ 500.000

→ 250.000

→ 125.000

→ ...

→ 1 (después de 20 pasos)

mmm y es siempre 1 como resultado porque queremos encontrar un resultado?

R: Exacto, lo entendiste perfecto.

El 1 representa **un solo elemento** – que es cuando **ya no puedes dividir más**.

### En búsqueda binaria:

Tienes 1.000 elementos



Divides hasta quedar con 1 solo elemento

Ese elemento es el que buscabas ✓

### En Merge Sort:

Divides el array hasta tener grupos de 1 elemento

Un elemento solo siempre está ordenado

Ahí empieza a unir ✓

## La lógica es:

¿Cuántas veces divido entre 2 hasta llegar a 1 elemento?  
Esa cantidad de divisiones = los pasos del algoritmo

Por eso  $\log_2(n)$  siempre termina en 1 – porque es el **caso base**, el punto donde el problema ya no se puede dividir mas.

---

En Merge Sort:

Divides el array hasta tener grupos de 1 elemento

Un elemento solo siempre está ordenado

Ahí empieza a unir 

eso de ahí es lo que vemos en algoritmos de ordenamiento? (dime si si o no, ya que estoy tomando notas de momento)

Sí — eso es exactamente lo que vemos en algoritmos de ordenamiento.

### 1.1.2 $O(n \log n)$ — ¿Por qué 33 pasos con 10 elementos?

| Notación      | 10 elementos | 1.000 elementos | 1.000.000 elementos |
|---------------|--------------|-----------------|---------------------|
| $O(1)$        | 1 paso       | 1 paso          | 1 paso              |
| $O(\log n)$   | 3 pasos      | 10 pasos        | 20 pasos            |
| $O(n)$        | 10 pasos     | 1.000 pasos     | 1.000.000 pasos     |
| $O(n \log n)$ | 33 pasos     | 10.000 pasos    | 20.000.000 pasos    |
| $O(n^2)$      | 100 pasos    | 1.000.000 pasos | 💩                   |

Es simplemente **n multiplicado por log n**:

```
n = 10
log n = 3

n log n = 10 × 3 = 30 ≈ 33 pasos
```

**Con n = 1.000**

```
n = 1.000
log n = 10

n log n = 1.000 × 10 = 10.000 pasos
```

**Con n = 1.000.000**

```
n = 1.000.000
log n = 20

n log n = 1.000.000 × 20 = 20.000.000 pasos
```





### 1.1.2.1 Explicación detallada (explicación concreta)

$O(n \log n)$  son DOS operaciones combinadas:

$n$  → recorre todos los elementos (como  $O(n)$ )  
 $\log n$  → por cada elemento, hace una operación logarítmica

$n$  → recorre todos los elementos (como  $O(n)$ )  
 $\log n$  → por cada elemento, hace una operación logarítmica

**La analogía:**

Imagina que tienes 10 personas en una fila y a cada persona le haces 3 preguntas:

$10 \text{ personas} \times 3 \text{ preguntas} = 30 \text{ interacciones}$

Eso es exactamente  $n \times \log n$ .

**¿Por qué Merge Sort es  $O(n \log n)$ ?**

$\log n$  → cuántas veces divides el array hasta llegar a grupos de 1 con 10 elementos = 3 divisiones

$n$  → en cada nivel de división, recorres todos los elementos para unirlos (merge)

**Log  $n$**  → cuantas veces divides el array hasta llegar a grupos de 1 con 10 elementos = 3 divisiones  
 **$n$**  → en cada nivel de división, recorres todos los elementos para unirlos (merge)

Visualmente:

Nivel 1: [5, 3, 1, 4, 2, 8, 7, 6, 9, 0] → recorre 10 elementos  
Nivel 2: [5,3,1,4,2] [8,7,6,9,0] → recorre 10 elementos  
Nivel 3: [5,3][1,4,2][8,7][6,9,0] → recorre 10 elementos

3 niveles  $\times$  10 elementos = **30 pasos** ✓

**El calculo:**

$n = 10$  →  $10 \times \log_2(10) = 10 \times 3.32 = 33$   
 $n = 1000$  →  $1000 \times \log_2(1000) = 1000 \times 9.96 = 9960 \approx 10.000$

### 1.1.2.2 Merge Sort - ¿Qué pasa en cada nivel?

Merge Sort divide el array a la mitad en cada nivel. Pero aunque se divida en pedazos mas chicos, **la suma de elementos siempre es 10.**

---

#### Nivel 1 – 1 grupo de 10:

```
[5, 3, 1, 4, 2, 8, 7, 6, 9, 0]  
→ 10 elementos en total
```

#### Nivel 2 – 2 grupos de 5:

```
[5, 3, 1, 4, 2] [8, 7, 6, 9, 0]  
→ 5 + 5 = 10 elementos en total
```



#### Nivel 3 — 4 grupos más chicos:

```
[5, 3] [1, 4, 2] [8, 7] [6, 9, 0]  
→ 2 + 3 + 2 + 3 = 10 elementos en total
```



### ¿Ves el patrón?

Los grupos se dividen, pero la suma siempre da 10. Entonces en **cada nivel** el algoritmo trabaja con los 10 elementos – ya sea en 1 grupo, 2 grupos o 4 grupos.

---

### El costo total:

```
Cuántos niveles hay   =  $\log n = 3$   
Elementos por nivel   =  $n = 10$   
Total                  =  $3 \times 10 = 30$  pasos
```

```
Cuántos niveles hay =  $\log n = 3$   
Elementos por nivel =  $n = 10$   
Total =  $3 \times 10 = 30$  pasos
```

### 1.1.2.3 ¿Cómo es el cálculo de ir dividiendo? + Explicación e ir comparando arrays + Empezar a unit en orden inverso (merge)

El calculo es simplemente ir dividiendo entre 2 hasta llegar a 1:

```
Nivel 1: 10 ÷ 2 = 5  
Nivel 2: 5 ÷ 2 = 2.5 ≈ 2  
Nivel 3: 2 ÷ 2 = 1 ✓
```

3 divisiones → 3 niveles →  $\log_2(10) = 3$

---

**Con 8 elementos es mas limpio:**

```
Nivel 1: 8 ÷ 2 = 4  
Nivel 2: 4 ÷ 2 = 2  
Nivel 3: 2 ÷ 2 = 1 ✓
```

3 niveles →  $\log_2(8) = 3$

---

**Con 16 elementos:**

```
Nivel 1: 16 ÷ 2 = 8  
Nivel 2: 8 ÷ 2 = 4  
Nivel 3: 4 ÷ 2 = 2  
Nivel 4: 2 ÷ 2 = 1 ✓
```

4 niveles →  $\log_2(16) = 4$

---

**La regla:**

```
¿Cuántas veces divido n entre 2 hasta llegar a 1?  
Esa cantidad = log n = cantidad de niveles
```

podrias explicarlo de nuevo y con los arrays mostrandolo?

### Con 8 elementos – más limpio para ver:

Array inicial: [5, 3, 1, 4, 2, 8, 7, 6]

#### Nivel 1 — divide en 2:

[5, 3, 1, 4]      [2, 8, 7, 6]

$8 \div 2 = 4$  elementos por grupo

#### Nivel 2 – divide en 4:

[5, 3]    [1, 4]    [2, 8]    [7, 6]

$4 \div 2 = 2$  elementos por grupo

#### Nivel 3 – divide en 8:

[5] [3] [1] [4] [2] [8] [7] [6]



$2 \div 2 = 1$  elemento por grupo ☒ — caso base

#### Ahora cuenta los niveles:

Nivel 1 → 8 elementos en total  
Nivel 2 → 8 elementos en total  
Nivel 3 → 8 elementos en total

3 niveles x 8 elementos = **24 pasos**

Y  $\log_2(8) = 3$  ☒

### Ahora empieza a unir (merge) en orden inverso:

Nivel 3: [5][3] → [3, 5]      [1][4] → [1, 4]      [2][8] → [2, 8]      [7][6] → [6, 7]  
Nivel 2: [3,5][1,4] → [1,3,4,5]      [2,8][6,7] → [2,6,7,8]  
Nivel 1: [1,3,4,5][2,6,7,8] → [1,2,3,4,5,6,7,8] ☒

1.1.2.4 Explicación del log (misma pregunta dicha de dos formas: ¿Cuántas veces dividido 8 entre 2 hasta llegar a 1? - ¿A que potencia elevo 2 para obtener 8?

.3 niveles × 8 elementos = 24 pasos  
Y  $\log_2(8) = 3$    
de donde sale ese log?

R: El log sale de contar cuantas veces dividiste entre 2.

Literalmente constante las divisiones:

$8 \div 2 = 4$  → división 1  
 $4 \div 2 = 2$  → división 2  
 $2 \div 2 = 1$  → división 3

Hiciste **3 divisiones** – eso es  $\log_2(8) = 3$ .

La conexión:

$\log_2(8)$  pregunta:  
¿A qué potencia elevo 2 para obtener 8?

$2^1 = 2$   
 $2^2 = 4$   
 $2^3 = 8$  

Respuesta: 3



O sea, son la misma pregunta dicha de dos formas:

| Forma      | Pregunta                                            |
|------------|-----------------------------------------------------|
| Dividiendo | ¿Cuántas veces dividido 8 entre 2 hasta llegar a 1? |
| Logaritmo  | ¿A qué potencia elevo 2 para obtener 8?             |

| Forma      | Pregunta                                            |
|------------|-----------------------------------------------------|
| Dividiendo | ¿Cuántas veces dividido 8 entre 2 hasta llegar a 1? |
| Logaritmo  | ¿A que potencia elevo 2 para obtener 8?             |

Las dos dan **3** — son exactamente lo mismo.

### 1.1.2.5 Se divide para ordenar y se vuelve a armar ordenado - ¿Para qué sirve? - ¿Por qué dividir para ordenar?

mmm caso de uso?  
para que sirve hacer eso?  
Ahora empieza a unir (merge) en orden inverso:

```
Nivel 3: [5][3] → [3, 5]      [1][4] → [1, 4]      [2][8] → [2, 8]
[7][6] → [6, 7]
Nivel 2: [3,5][1,4] → [1,3,4,5]    [2,8][6,7] → [2,6,7,8]
Nivel 1: [1,3,4,5][2,6,7,8] → [1,2,3,4,5,6,7,8] ✓
```

y lo que se hace es dividirse y como que se vuelve a armar?

[fíjate que ahí se van ordenando]

R: Si exacto – se divide para ordenar y se vuelve a armar ordenado.

#### ¿Para qué sirve?

Para ordenar una lista de datos. Caso real:

Tienes 1.000.000 de productos en PREF  
Necesitas mostrarlos ordenados por precio

Sin algoritmo eficiente → 1.000.000.000.000 pasos ☹️  
Con Merge Sort → 20.000.000 pasos ✓

#### ¿Por qué dividir para ordenar?

Porque ordenar 1 elemento es trivial – siempre esta ordenado. Entonces la idea es:

Divide hasta tener grupos de 1  
→ cada grupo ya está "ordenado"  
→ ahora une los grupos comparando

#### El truco esta en el merge – ¿Cómo une ordenado?

Cuando une dos grupos, compara el primero de cada uno y toma el menor:

[3, 5] y [1, 4]

¿3 o 1? → toma 1 → [1]  
¿3 o 4? → toma 3 → [1, 3]  
¿5 o 4? → toma 4 → [1, 3, 4]  
queda 5 → [1, 3, 4, 5] ✓

#### El resumen del proceso completo:

DIVIDE → hasta grupos de 1  
COMPARA → el primero de cada grupo  
UNE → siempre en orden

**DIVIDE** → hasta grupos de 1  
**COMPARA** → el primero de cada grupo  
**UNE** → siempre en orden

### 1.1.2.6 Resolviendo de si siempre llega a grupo de 1 + Como ocurre el proceso de merge + ¿Cómo funciona el ordenamiento de nivel 2 a nivel 1?

El truco está en el merge — ¿cómo une ordenado?  
Cuando une dos grupos, compara el primero de cada uno y toma el menor:

```
[3, 5] y [1, 4]

¿3 o 1? → toma 1 → [1]
¿3 o 4? → toma 3 → [1, 3]
¿5 o 4? → toma 4 → [1, 3, 4]
queda 5      → [1, 3, 4, 5] ✓
```

El resumen del proceso completo:

```
DIVIDE → hasta grupos de 1
COMPARA → el primero de cada grupo
UNE → siempre en orden
```

pero si siempre tendría que llegar a grupos de 1 o no?  
Mostrar menos

**R:** Si, siempre llega a grupos de 1 – eso es el caso base.  
Pero el ordenamiento real ocurre en el merge, no en la división.

#### La división solo prepara el terreno:

```
[5, 3, 1, 4]
→ divide hasta grupos de 1
[5] [3] [1] [4]
```

Aquí no hay nada ordenado todavía – solo grupos de 1.

#### El merge es donde ocurre la magia:

```
[5] y [3] → compara → [3, 5] ✓ ordenado
[1] y [4] → compara → [1, 4] ✓ ordenado

[3, 5] y [1, 4] → compara de a uno → [1, 3, 4, 5] ✓ ordenado
```

#### Tenemos dos grupos ya ordenados:

```
[3, 5] y [1, 4]
```

Cada grupo tiene un “puntero” apuntando al primer elemento:

```
[3, 5] → puntero en 3
[1, 4] → puntero en 1
```

### Comparación 1:

3 vs 1 → 1 es menor → toma 1  
resultado: [1]

[3, 5] → puntero sigue en 3  
[1, 4] → puntero avanza a 4

### Comparación 2:

3 vs 4 → 3 es menor → toma 3  
resultado: [1, 3]

[3, 5] → puntero avanza a 5  
[1, 4] → puntero sigue en 4

### Comparación 3:

5 vs 4 → 4 es menor → toma 4  
resultado: [1, 3, 4]



[3, 5] → puntero sigue en 5  
[1, 4] → grupo vacío ✓

### Queda 5 solo – se agrega directo:

resultado: [1, 3, 4, 5] ✓

### La regla del puntero:

Compara el primero de cada grupo  
→ toma el menor  
→ avanza el puntero de ese grupo  
→ repite hasta vaciar ambos grupos

Compara el primero de cada grupo  
->toma el menor  
->avanza el punto de ese grupo  
-> repite hasta vaciar ambos grupos

mmm caso de uso?  
para que sirve hacer eso?  
Ahora empieza a unir (merge) en orden inverso:

Nivel 3: [5][3] → [3, 5]      [1][4] → [1, 4]      [2][8] → [2, 8]  
[7][6] → [6, 7]  
Nivel 2: [3,5][1,4] → [1,3,4,5]      [2,8][6,7] → [2,6,7,8]  
Nivel 1: [1,3,4,5][2,6,7,8] → [1,2,3,4,5,6,7,8] ✓

y lo que se hace es dividirse y como que se vuelve a armar?

### ¿Y para este caso cómo funciona el ordenamiento de nivel 2 a nivel 1?

R:

Tenemos dos grupos de Nivel 2:



[1, 3, 4, 5] y [2, 6, 7, 8]

Puntero en el primero de cada grupo:

[1, 3, 4, 5] → puntero en 1  
[2, 6, 7, 8] → puntero en 2

### Comparación 1:

1 vs 2 → 1 es menor → toma 1  
resultado: [1]



[1, 3, 4, 5] → puntero avanza a 3  
[2, 6, 7, 8] → puntero sigue en 2

### Comparación 2:

3 vs 2 → 2 es menor → toma 2  
resultado: [1, 2]



[1, 3, 4, 5] → puntero sigue en 3  
[2, 6, 7, 8] → puntero avanza a 6

### Comparación 3:

3 vs 6 → 3 es menor → toma 3  
resultado: [1, 2, 3]



[1, 3, 4, 5] → puntero avanza a 4  
[2, 6, 7, 8] → puntero sigue en 6

### Comparación 4:

4 vs 6 → 4 es menor → toma 4  
resultado: [1, 2, 3, 4]



[1, 3, 4, 5] → puntero avanza a 5  
[2, 6, 7, 8] → puntero sigue en 6

### Comparación 5:

5 vs 6 → 5 es menor → toma 5  
resultado: [1, 2, 3, 4, 5]

[1, 3, 4, 5] → grupo vacío ✓  
[2, 6, 7, 8] → puntero sigue en 6

Queda [6, 7, 8] — se agregan directo:

resultado: [1, 2, 3, 4, 5, 6, 7, 8] ✓

### La clave:

Cuando un grupo se vacía, el resto del otro grupo se agrega directo sin comparar – porque ya está ordenado.

## ¿Por qué necesita llegar a grupos de 1?

Porque es el único tamaño que garantiza que el grupo ya está ordenado – un solo elemento no puede estar desordenado.

[3, 5] → ¿está ordenado? no sabes  
[3] → ¿está ordenado? siempre ☒

Si intentaras mergear sin dividir hasta 1, no podrías garantizar que los grupos de entrada ya están ordenados.

## 1.2 Profundización – Complejidad Algorítmica + notaciones principales

### 1.2.1 Complejidad Algorítmica – ¿Qué es n?

#### ¿Qué es Complejidad Algorítmica?

Cuando escribes un algoritmo, siempre surge la pregunta: **¿Qué tan eficiente es?**

No se mide en segundos porque eso depende del computador – una CPU de 2024 va a correr el mismo código mas rápido que una de 2010. Entonces necesitamos una medida **independiente del hardware**.

La complejidad algorítmica mide **cuantos pasos ejecuta un algoritmo** según el tamaño del input **n**.

---

#### ¿Qué es n?

Es la cantidad de datos que recibe el algoritmo.

```
n = 10    → tienes 10 elementos  
n = 1000 → tienes 1000 elementos
```



Mientras mas grande es n, mas paso necesita el algoritmo. La pregunta es **cuanto más**.

### 1.2.2 ¿Qué es la Notación Big O? + Notaciones principales

Es la forma estándar de **expresar la complejidad**. Se escribe **O(algo)** y describe el **peor caso posible** – cuantos pasos necesita el algoritmo cuando n crece al infinito.

```
O(1)      → los pasos no crecen  
O(n)      → los pasos crecen igual que n  
O(n²)     → los pasos crecen al cuadrado
```



```
O(1) → Los pasos no crecen  
O(n) → Los pasos crecen igual que n  
O(n²) → Los paos crecen al cuadrado
```

Las notaciones principales

1.2.2.1 O (1) – Constante

Sin importar cuántos datos hay, **siempre ejecuta el mismo número de pasos.**

| Algoritmo                              | Descripción                               |
|----------------------------------------|-------------------------------------------|
| Acceder a un elemento de array         | <code>\$arr[5]</code> siempre es 1 paso   |
| Insertar al final de una pila          | Siempre agrega en la misma posición       |
| Verificar si un número es par          | <code>n % 2</code> siempre es 1 operación |
| Obtener el primer elemento de una cola | Siempre está en la misma posición         |
| Hash table lookup                      | Buscar por clave siempre es directo       |

| Algoritmo                              | Descripcion                               |
|----------------------------------------|-------------------------------------------|
| Acceder a un elemento de array         | <code>\$arr[5]</code> siempre es 1 paso   |
| Insertar al final de una pila          | Siempre agrega en la misma posición       |
| Verificar si un número es par          | <code>N % 2</code> siempre es 1 operación |
| Obtener el primer elemento de una cola | Siempre esta en la misma posición         |
| Hash table lookup                      | Buscar por clave siempre es directo       |

1.2.2.2 O (log n) – logarítmica

Cada paso **descarta la mitad** de los datos restantes. Es extremadamente eficiente.

1.000.000 elementos → solo ~20 pasos

| Algoritmo                        | Descripción                                   |
|----------------------------------|-----------------------------------------------|
| Búsqueda binaria                 | Divide el array a la mitad en cada paso       |
| Búsqueda en árbol binario (BST)  | Descarta la mitad del árbol en cada nivel     |
| Algoritmo de Euclides (MCD)      | Reduce el problema a la mitad cada vez        |
| Búsqueda en diccionario ordenado | Abre por la mitad y descarta                  |
| Potenciación rápida              | Divide el exponente a la mitad cada iteración |

| Algoritmo                        | Descripcion                                   |
|----------------------------------|-----------------------------------------------|
| Busqueda binaria                 | Divide el array a la mitad en cada pao        |
| Busqueda en árbol binario (BST)  | Descarta la mitad del árbol en cada nivel     |
| Algoritmo de Euclides (MCD)      | Reduce el problema a la mitad cada vez        |
| Busqueda en diccionario ordenado | Abre por la mitad y descarta                  |
| Potenciacion rápida              | Divide el exponente a la mitad cada iteración |

### 1.2.2.3 $O(n)$ – Lineal

Los pasos crecen **igual que  $n$** . El doble de datos = el doble de pasos.

| Algoritmo                     | Descripción                        |
|-------------------------------|------------------------------------|
| Recorrer un array             | Un paso por cada elemento          |
| Búsqueda lineal               | Revisa uno por uno hasta encontrar |
| Contar elementos en una lista | Recorre toda la lista              |
| Encontrar el máximo/mínimo    | Revisa todos los elementos         |
| Copiar un array               | Copia elemento por elemento        |

| Algoritmo                     | descripción                        |
|-------------------------------|------------------------------------|
| Recorrer un array             | Un paso por cada elemento          |
| Busqueda lineal               | Revisa uno por uno hasta encontrar |
| Contar elementos en una lista | Recorre toda la lista              |
| Encontrar el máximo/mínimo    | Revisa todos los elementos         |
| Copiar un array               | Copia elemento por elemento        |

### 1.2.2.4 $O(n \log n)$ – Lineal-logarítmica

Mas lento que  $O(n)$  pero **mucho mejor que  $O(n^2)$** . Es el costo típico de los buenos algoritmos de ordenamiento.

| Algoritmo                  | Descripción                          |
|----------------------------|--------------------------------------|
| Merge Sort                 | Divide y une recursivamente          |
| Quick Sort (caso promedio) | Divide por pivote recursivamente     |
| Heap Sort                  | Ordena usando una estructura heap    |
| Tim Sort                   | Usado por Python y Java internamente |
| Intro Sort                 | Usado por C++ en la STL              |

| Algoritmo                  | Descripcion                           |
|----------------------------|---------------------------------------|
| Merge Sort                 | Divide y une recursivamente           |
| Quick Sort (caso promedio) | Divide por pivote recursivamente      |
| Heap Sort                  | Ordena usando una estructura heap     |
| Tim Sort                   | Usando por Python y Java internamente |
| Intro Sort                 | Usado por C++ en la STL               |

1.2.2.5 O(n²) — Cuadrática

Un loop dentro de otro. Se vuelve inutilizable con datos grandes.

1.000 elementos → 1.000.000 pasos 🤖

| Algoritmo                            | Descripción                                   |
|--------------------------------------|-----------------------------------------------|
| Bubble Sort                          | Compara cada par adyacente n veces            |
| Selection Sort                       | Busca el mínimo n veces                       |
| Insertion Sort                       | Inserta cada elemento en su posición correcta |
| Comparar todos los pares de un array | Dos loops anidados                            |
| Multiplicación de matrices simple    | Tres loops anidados → O(n³)                   |

| Algoritmo                            | Descripcion                                   |
|--------------------------------------|-----------------------------------------------|
| Bubble Sort                          | Compara cada par adyacente n veces            |
| Selection Sort                       | Busca el mínimo n veces                       |
| Insertion Sort                       | Inserta cada elemento en su posición correcta |
| Comparar todos los pares de un array | Dos loops anidados                            |
| Multiplicados de matrices simple     | Tres loops anidados → O(n³)                   |

## 1.3 Casos de uso por notación

### 1.3.1 $O(1)$ – Constante

## X. Índice — Algoritmos y Estructuras de Datos + lo que faltaría

### 1. Complejidad Algorítmica

- 1.1 ¿Qué es un algoritmo?
- 1.2 ¿Qué es complejidad algorítmica?
- 1.3 Notación Big O
- 1.4  $O(1)$  — Constante
- 1.5  $O(\log n)$  — Logarítmica
- 1.6  $O(n)$  — Lineal
- 1.7  $O(n \log n)$  — Lineal-logarítmica
- 1.8  $O(n^2)$  — Cuadrática
- 1.9 Tabla comparativa

### 2. Algoritmos de Ordenamiento

- 2.1 Bubble Sort —  $O(n^2)$
- 2.2 Selection Sort —  $O(n^2)$
- 2.3 Insertion Sort —  $O(n^2)$
- 2.4 Merge Sort —  $O(n \log n)$
- 2.5 Quick Sort —  $O(n \log n)$

### 3. Recursión

- 3.1 ¿Qué es recursión?
- 3.2 Caso base
- 3.3 Llamada recursiva
- 3.4 Ejemplo — Factorial
- 3.5 Conexión con Merge Sort

### 4. Estructuras de Datos — Árboles

- 4.1 ¿Qué es un nodo?
- 4.2 ¿Qué es una arista?
- 4.3 ¿Qué es un árbol?
- 4.4 Partes de un árbol (raíz, hijo, padre, hoja)
- 4.5 Árbol binario
- 4.6 Árbol binario de búsqueda (BST)
- 4.7 Complejidad en BST —  $O(\log n)$
- 4.8 Insertar un nodo
- 4.9 Eliminar un nodo
- 4.10 Árbol balanceado vs desbalanceado

### 5. Recorridos de Árboles y Grafos

- 5.1 ¿Qué es recorrer?
- 5.2 DFS — Depth First Search
- 5.3 BFS — Breadth First Search
- 5.4 Diferencias entre DFS y BFS

### 6. Grafos

- 6.1 ¿Qué es un grafo?
- 6.2 Grafo no dirigido
- 6.3 Grafo dirigido
- 6.4 Grafo ponderado
- 6.5 Ciclos
- 6.6 BFS en grafos
- 6.7 DFS en grafos

### 7. Algoritmos de Camino más Corto

- 7.1 ¿Qué es el camino más corto?
  - 7.2 Dijkstra — explicación
  - 7.3 Dijkstra — paso a paso
  - 7.4 Fórmula de Dijkstra
  - 7.5 Bellman-Ford (pesos negativos)
-



eso sería lo fundamental que se pasaría en un ramo por ejemplo de estructura o complejidad de algoritmos?

### Lo que cubrimos — es la base sólida ✓

Es lo que se ve en el primer semestre de un ramo típico de Estructuras de Datos en cualquier universidad.

### Lo que faltaría para un ramo completo:

| Tema                    | Por qué importa                                  |
|-------------------------|--------------------------------------------------|
| Listas enlazadas        | La estructura más básica antes de árboles        |
| Pilas y colas           | Las mencionamos pero no las explicamos formales  |
| Tablas hash             | Una de las estructuras más usadas en la práctica |
| Árboles AVL/ Rojo-Negro | Árboles que se auto-balancean                    |
| Heaps                   | Estructura para colas de prioridad               |
| Programación dinámica   | Optimización de algoritmos recursivos            |
| Algoritmos greedy       | Tomar la mejor decisión local en cada paso       |
| Ordenamiento externo    | Cuando los datos no caben en memoria RAM         |

### La realidad según las universidades chilenas:

En INACAP, UMAG o Universidad Mayor un ramo de Estructuras de Datos cubre exactamente lo que vimos más **listas enlazadas, pilas, colas y tablas hash**. Los árboles AVL y programación dinámica generalmente aparecen en un segundo ramo más avanzado.

En universidades más exigentes como PUC o USM agregan **programación dinámica y algoritmos greedy** en el mismo ramo.

### Para lo que tú necesitas — como dev freelance y estudiando ingeniería:

Lo que vimos hoy ya te pone muy por encima del promedio. La mayoría de los devs en producción nunca implementan un AVL — pero sí necesitan entender Big O para no escribir código  $O(n^2)$  sin darse cuenta.